

Проблема блокирующего кода

Почти весь код, с которым вы сталкивались, скорее всего, был **синхронным**.

Что это значит?

Синхронный код — это код, который выполняется строго последовательно, команда за командой. Новая команда не начнет выполняться, пока предыдущая полностью не завершится.

Аналогия: Повар-однозадачник

Представьте себе повара, который получил заказ на обед: сварить суп, пожарить стейк и приготовить кофе.

Наш повар работает **синхронно**, поэтому его действия будут такими:

1. **Задача №1: Суп.** Он ставит кастрюлю с водой на плиту. И всё. Он стоит и **ждёт**, пока вода закипит. Он не может в это время резать овощи для стейка или засыпать зерна в кофемашину. Он просто ждёт.
2. Вода закипела. Он добавляет ингредиенты и снова **ждёт**, пока суп сварится. Всё его внимание приковано к супу.
3. Суп готов. Повар отставляет его в сторону. Только теперь, когда первая задача на 100% завершена, он переходит к следующей.
4. **Задача №2: Стейк.** Он берет мясо, кладет на сковороду и **ждёт**, пока оно обжарится с одной стороны. Затем переворачивает и снова **ждёт**.
5. Стейк готов.
6. **Задача №3: Кофе.** Он включает кофемашину и **ждёт**, пока напиток приготовится.

Главная проблема этого подхода — огромное количество времени, потраченного на **ожидание**. Большую часть времени повар не работал, а простаивал, хотя на кухне было много других дел.

Мы увидели, что наша программа много времени тратила на ожидание, выполняя "блокирующие операции". Чтобы понять, когда это становится настоящей проблемой, важно научиться различать два фундаментальных типа задач в программировании: **CPU-bound** и **I/O-bound**.

CPU-bound (Вычислительные задачи)

Представьте, что вы попросили программу выполнить самый сложный математический расчет, например, заархивировать большой файл или обработать видео.

***CPU-bound** задачи — это задачи, скорость выполнения которых ограничена мощностью вашего процессора (CPU).*

Примеры CPU-bound задач:

- Сложные математические вычисления.
- Архивация и разархивация данных.
- Обработка изображений и видео.
- Криптографические операции.

I/O-bound (Задачи ввода-вывода)

Теперь вспомним наш пример с `time.sleep()` и повара, который ждет. В этих случаях процессор, наоборот, почти не работал. Он просто простаивал, ожидая, пока что-то произойдет во внешнем мире.

Примеры I/O-bound задач:

- Работа с сетью: HTTP-запросы к сайтам, скачивание файлов.
- Работа с диском: чтение или запись файлов.
- Работа с базами данных: выполнение SQL-запросов.

Для чего же нужна асинхронность?

Асинхронность — это, по своей сути, искусство **эффективного управления ожиданием**.

Вернемся к нашему повару. Синхронный подход заставлял его стоять и ждать, пока закипит вода. Асинхронный подход позволил бы ему, поставив воду на плиту, не ждать, а **переключиться** на другую задачу — например, начать нарезать овощи. Как только вода закипит (поступит внешний сигнал), он сможет вернуться к супу.

А вот для CPU-bound задач асинхронность на одном ядре процессора почти бесполезна. Если повар занят очень сложным рецептом, требующим всей его концентрации (аналог 100% загрузки CPU), то переключение на другую сложную задачу ему не поможет, а только собьет с толку и замедлит общую работу. Для таких задач существуют другие подходы, например, многопроцессорность.

Основная мысль этого урока: асинхронность не делает ваш процессор быстрее. Она позволяет вашей программе не простаивать впустую во время ожидания данных, а выполнять другую полезную работу.

Ключевые понятия `asyncio`

В предыдущем уроке мы выяснили, что главная проблема синхронного кода — это бесполезное ожидание во время I/O-операций. Наш повар просто стоял у плиты, а наша программа "спала", хотя процессор мог бы делать что-то полезное.

Так как же это исправить? Как заставить программу переключаться на другие дела, вместо того чтобы ждать?

Ответ кроется в концепции **кооперативной многозадачности**.

Аналогия: Жонглер

Давайте забудем про нашего медленного повара и представим себе профессионального жонглера.

- **Жонглер** — это наша асинхронная система управления (позже мы узнаем, что это **Цикл событий**).
- **Шарики** — это наши задачи (например, "скачать файл", "обработать данные").

Синхронный повар брал одну задачу (один шарик), доводил ее до конца и только потом брал следующую.

1. Он подбрасывает первый шарик (запускает **задачу №1**).
2. Но он **не ждет**, пока шарик долетит до высшей точки и упадет! Это было бы неэффективно.
3. Как только первый шарик взлетел, жонглер тут же, не теряя ни мгновения, подбрасывает второй шарик (запускает **задачу №2**).
4. Затем сразу же третий шарик (**задача №3**).
5. Когда первый шарик, долетев до пика, начинает падать (когда **задача №1** дошла до точки ожидания, например, отправила запрос в сеть и ждет ответа), жонглер ловит его и, если нужно, снова подбрасывает (продолжает работу над задачей).

Ключевая идея: жонглер никогда не простаивает. Он постоянно переключает свое внимание между шариками, уделяя каждому ровно столько времени, сколько нужно, чтобы он не упал. Благодаря этому в воздухе одновременно находятся несколько шариков, хотя в руке у жонглера в каждый конкретный момент — только один.

Почему "кооперативная"?

Этот вид многозадачности называется **кооперативной**, потому что задачи как бы "договариваются" и вежливо уступают друг другу управление.

Наша программа будет выполнять задачу до тех пор, пока сама задача не скажет: "Всё, я отправила запрос в сеть, теперь мне нужно подождать. Можешь пока поработать над чем-нибудь другим". В этот момент она "кооперирует", добровольно отдавая управление обратно "жонглеру", чтобы тот мог переключиться на другую задачу.

Итак, мы поняли, что асинхронная программа похожа на жонглера, который управляет несколькими шариками одновременно. В библиотеке `asyncio` эта система построена на трех ключевых компонентах, "трех китах", на которых все держится.

1. Цикл событий (Event Loop) — Это наш жонглер

Цикл событий — это ядро любой асинхронной программы. Это бесконечный цикл, который

выступает в роли менеджера или диспетчера задач.

Именно он:

- Следит за всеми активными задачами.
- Решает, какую задачу запустить следующей.
- Запускает задачу и ждет, пока она добровольно не вернет управление (скажет: "я уйду в режим ожидания").
- Как только одна задача отдает управление, цикл событий тут же находит другую, готовую к работе, и передает управление ей.

Вы не будете видеть цикл событий постоянно, но он всегда работает "под капотом", дирижируя всем процессом. **Event Loop** — это и есть наш жонглер.

2. Корутина (Coroutine) — Это инструкция для шарика

Простая функция в Python выполняется от начала до конца без остановок. Но для асинхронной магии нам нужны "особенные" функции.

Корутина (или "сопрограмма") — это функция, выполнение которой можно приостановить в определенном месте, а затем, спустя какое-то время, возобновить с того же самого места.

Представьте, что корутина — это инструкция по выполнению одного броска шарика. Она содержит в себе логику: "подбросить шарик, а пока он летит, отдать управление жонглеру".

Сама по себе корутина ничего не делает. Вызов такой функции просто создает объект корутины, но не запускает ее код. Это просто "рецепт" или "план" действий.

3. Задача (Task) — Это шарик в полете

Итак, у нас есть жонглер (Цикл событий) и инструкция (Корутина). Как их связать? Как

заставить жонглера действительно начать работать с шариком?

Задача (Task) — это специальный "контейнер" или "обертка" вокруг корутины, которая позволяет Циклу событий управлять ею.

Когда мы хотим, чтобы наша корутина начала выполняться, мы не вызываем ее напрямую. Мы говорим циклу событий: "Эй, вот корутина, пожалуйста, создай для нее Задачу и начни выполнять ее, когда сможешь".

Именно Задача отслеживает состояние корутины (работает, ждет, завершена) и позволяет циклу событий эффективно ею управлять. Task — это и есть наш шарик, который находится в процессе жонглирования.

Как они связаны?

- 1. Мы пишем логику в Корутике.**
- 2. Мы "оборачиваем" корутину в Задачу, чтобы ее можно было запустить.**
- 3. Цикл событий подхватывает эту Задачу и начинает ее выполнять, переключаясь между ней и другими задачами.**

Цикл событий	"Жонглер", который всем управляет.
Корутина	"Рецепт" или инструкция, описывающая, что делать.
Задача	"Шарик в полете", объект, который отслеживает процесс.

Создание и запуск корутин

корутины — функции, которую можно приостанавливать. Пришло время научиться создавать такие функции в Python.

Синтаксис `async def`

Чтобы объявить функцию как корутину (или, если точнее, как "функцию-корутину", которая создает корутину), используется очень простое и понятное ключевое слово `async` перед привычным `def`.

Сравните: **Обычная (синхронная) функция:** **Асинхронная функция (корутина):**

```
def regular_function():  
    return "Я обычная функция!"
```

```
async def coroutine_function():  
    return "Я особенная функция-корутина!"
```

Всего одно слово `async` магическим образом меняет поведение функции.

Главное отличие: что происходит при вызове?

И вот тут кроется фундаментальная разница.

Когда вы вызываете **обычную функцию**, ее код **немедленно выполняется**, и вы сразу получаете результат:

```
result = regular_function()  
print(result)
```

```
# Вывод: Я обычная функция!
```

А что произойдет, если вызвать функцию, объявленную с `async def`?

```
result = coroutine_function()  
  
print(result)
```

Если вы запустите этот код, вы **не увидите** строку "Я особенная функция-корутина!". Вместо этого консоль покажет что-то вроде:

```
<coroutine object coroutine_function at 0x10e82c740>
```

Почему так?

Потому что вызов функции, объявленной с `async def`, **не выполняет ее код**. Он лишь создает и возвращает специальный объект — **объект корутины**.

Можно думать об этом так:

- Вызов обычной функции — это как прийти в кафе и сразу **получить** готовый кофе.
- Вызов `async`-функции — это как получить **чек с заказом** на кофе. У вас есть подтверждение заказа, но самого напитка еще нет. Чтобы его получить, этот чек нужно отдать "баристе" (нашему циклу событий), который его приготовит.

Ключевой вывод: `async def` — это синтаксис для создания корутин. Вызов такой функции не запускает код, а лишь возвращает "план" или "рецепт" его выполнения в виде объекта корутины.

Давайте на практике посмотрим, что возвращает вызов функции, объявленной с `async def`. Мы создадим простейшую корутину, которая просто печатает сообщение, а затем вызовем ее так, как привыкли вызывать обычные функции.

Пример кода

Скопируйте, запустите и внимательно посмотрите на результат.

```
# Для работы с асинхронностью нам понадобится эта библиотека.
# Пока просто импортируем ее, ее роль станет ясна чуть позже.
import asyncio

# 1. Объявляем асинхронную функцию с помощью "async def"
async def my_first_coroutine():
    print("Привет, я корутина! Мой код начал выполняться.")

# --- Основная часть программы ---

print("Вызываем корутину...")

# 2. Вызываем ее как обычную функцию
coroutine_object = my_first_coroutine()

print("Корутина вызвана.")
print("Тип возвращенного объекта:", type(coroutine_object))
print("Сам объект:", coroutine_object)
```

Что вы увидите в консоли?

Ожидаемый вывод будет таким:

codeCode

```
Вызываем корутину...
Корутина вызвана.
Тип возвращенного объекта: <class 'coroutine'>
Сам объект: <coroutine object my_first_coroutine at 0x...>
```

(Точный адрес объекта 0x... у вас будет другой)

Разбор полетов

Самое важное, на что нужно обратить внимание:

- **Сообщение не напечаталось!** Строка `print("Привет, я корутина!...")` так и не была выполнена. Это доказывает, что код внутри `async def` функции **не запускается** при ее обычном вызове.

- **Вернулся специальный объект.** Как и говорилось в теории, мы получили не результат работы функции, а объект типа `<class 'coroutine'>`. Это тот самый "чек с заказом", который мы теперь держим в переменной `coroutine_object`.

Это поведение лежит в основе всей асинхронности. Мы создали "план действий", но еще не передали его "исполнителю" (Циклу событий).

Итак, у нас есть объект корутины — "чек с заказом". Но как заставить одну корутину дожждаться выполнения другой? Например, наша главная корутина `cook_dinner()` должна вызвать и дожждаться завершения корутины `boil_water()`.

Для этого в Python существует специальный оператор — `await`.

`await` (в переводе "ожидать") — это ключевое слово, которое используется для приостановки выполнения текущей корутины до тех пор, пока не завершится другая корутина (или другая "ожидаемая" операция).

Как работает `await`?

Представьте, что наш код дошел до такой строки внутри одной корутины:

```
result = await some_other_coroutine()
```

В этот момент происходит магия кооперативной многозадачности:

1. **Приостановка:** Текущая корутина **приостанавливает** свое выполнение. Она не продолжает работу дальше.
2. **Передача управления:** Управление передается обратно **Циклу событий** (нашему "жонглеру").
3. **Постановка новой задачи:** Корутина как бы говорит Циклу событий: "Пожалуйста, выполни `some_other_coroutine()`, а я пока подожду. Пока ты ее выполняешь, можешь заняться другими делами, если они у тебя есть".
4. **Возобновление:** Когда `some_other_coroutine()` наконец завершит свою работу и

вернет результат, Цикл событий "разбудит" нашу исходную корутину, вернет ей этот результат (который запишется в переменную `result`), и она продолжит свое выполнение со следующей строки.

Аналогия с поваром 2.0

Давайте вернемся к нашему повару, но теперь он — асинхронный.

Его главная задача `cook_dinner()` — приготовить ужин.
Один из шагов — вскипятить воду (`boil_water()`).

Когда он доходит до шага `await boil_water()`, он:

- Ставит кастрюлю с водой на плиту (запускает операцию).
- **И не стоит над ней!** Он говорит: "Отлично, вода греется, это займет время. А пока я пойду нарежу овощи для салата (другая задача)".
- Когда вода закипит (операция завершилась), он получит сигнал и вернется к работе с кастрюлей.

`await` — это и есть тот самый момент, когда повар **переключается** с ожидания на другую полезную работу.

Важное правило

Ключевое слово `await` можно использовать **только внутри функции, объявленной с `async def`**. Если вы попытаетесь использовать его в обычной синхронной функции, вы получите ошибку `SyntaxError`. Это логично: нельзя "приостановить" то, что по своей природе не умеет приостанавливаться.

Ключевой вывод: `await` — это точка "паузы" и "кооперации". Это механизм, с помощью которого корутина добровольно отдает управление, позволяя программе эффективно использовать время ожидания.

Теория — это хорошо, но пора увидеть `await` в действии. Чтобы это сделать, нам нужна асинхронная операция, которую можно "подождать".

Вы уже знакомы с блокирующей функцией `time.sleep()`, которая "замораживает" всю программу. В мире `asyncio` есть ее "вежливый" аналог — `asyncio.sleep()`.

В чем разница?

- `time.sleep(N)` говорит: "Остановить **всю программу** на N секунд". Это блокирующая операция.
- `await asyncio.sleep(N)` говорит: "Приостановить выполнение **только текущей корутины** на N секунд и отдать управление обратно Циклу событий, чтобы он мог поработать над другими задачами". Это неблокирующая, кооперативная операция.

`asyncio.sleep()` возвращает объект корутины, поэтому ее всегда нужно использовать с `await`.

Первый работающий пример

Давайте напишем полноценную асинхронную программу. Она будет состоять из двух корутин:

1. `wait_and_print()`: будет ждать 2 секунды, а затем печатать сообщение.
2. `main()`: будет запускать и ожидать завершения `wait_and_print()`.

```
import asyncio
import time

async def wait_and_print():
    print("Начинаю долгое ожидание...")

    # Вот она, точка приостановки!
    # Код "замрет" здесь, но не заблокирует программу.
    await asyncio.sleep(2)

    print("...ожидание завершено!")

async def main():
    print("Программа запущена.")
    start_time = time.time()

    # Вызываем и ЖДЕМ (await), пока wait_and_print не завершится.
    await wait_and_print()

    end_time = time.time()
    print(f"Программа завершена за {end_time - start_time:.2f} секунд.")

# asyncio.run() - это специальная функция, которая создает
# Цикл событий, запускает в нем нашу главную корутину 'main'
# и ждет ее завершения. Это точка входа в асинхронный мир.
asyncio.run(main())
```

Что вы увидите в консоли?

```
Программа запущена.
Начинаю долгое ожидание...
(здесь будет пауза примерно в 2 секунды)
...ожидание завершено!
Программа завершена за 2.00 секунд.
```

Разбор полетов

1. `asyncio.run(main())` запускает нашу главную корутину `main`.
2. Выполняется первая строка `main`, печатается "Программа запущена".
3. Программа доходит до строки `await wait_and_print()`. Она вызывает `wait_and_print` и ждет ее.

4. Выполнение переходит внутрь `wait_and_print`. Печатается "Начинаю долгое ожидание...".
5. Программа доходит до `await asyncio.sleep(2)`. В этот момент корутина `wait_and_print` **приостанавливается** и отдает управление циклу событий с просьбой "разбудить" ее через 2 секунды.
6. Цикл событий ждет 2 секунды (так как других задач у него нет).
7. Через 2 секунды он **возобновляет** работу `wait_and_print` с того места, где она остановилась.
8. Печатается "...ожидание завершено!", и `wait_and_print` завершается.
9. Корутина `main` дождалась своего `await` и продолжает работу.
10. Печатается финальное сообщение о времени выполнения.

Поздравляем! Вы только что написали и поняли свою первую асинхронную программу. На следующем шаге мы закрепим, как `asyncio.run()` связывает все это воедино.

Мы научились создавать корутины с помощью `async def` и приостанавливать их с помощью `await`. Но остался один важный вопрос: что заставляет всю эту систему работать? Кто создает того самого "жонглера" (Цикл событий) и передает ему нашу самую первую, главную корутину?

Ответ — функция `asyncio.run()`.

`asyncio.run(coroutine)` — это высокоуровневая функция, которая служит основной точкой входа для запуска асинхронной программы.

Можно думать о ней как о **ключе зажигания** для вашего асинхронного "двигателя". Ваш Python-скрипт начинается как обычная, синхронная программа. И в тот момент, когда исполнение доходит до `asyncio.run()`, вы поворачиваете ключ и запускаете весь асинхронный мир.

Что `asyncio.run()` делает для вас "под капотом"?

Когда вы вызываете `asyncio.run(main())`, она выполняет несколько важных действий в строгом порядке:

1. **Создает новый Цикл событий.** Она сама создает того самого "менеджера" или "жонглера", который будет всем управлять.
2. **Запускает вашу корутину.** Она берет объект корутины, который вы ей передали (в нашем случае `main()`), и дает Циклу событий команду: "Начинай выполнять вот это".
3. **Работает до полного завершения.** Цикл событий будет работать, переключая задачи, пока самая первая, главная корутина (`main`) полностью не завершит свое выполнение.
4. **Аккуратно все закрывает.** Как только `main` завершена, `asyncio.run()` автоматически останавливает Цикл событий и выполняет всю необходимую очистку ресурсов.

До появления `asyncio.run()` (в Python 3.7) разработчикам приходилось выполнять все эти шаги вручную, что было довольно громоздко. Эта функция — современный, простой и надежный способ запустить асинхронный код.

Мост между двумя мирами

`asyncio.run()` — это **мост** между синхронным и асинхронным миром.

- Ваш `.py` файл начинается **в синхронном мире**.
- `asyncio.run()` переносит вас **в асинхронный мир**, где правят `async` и `await`.
- Когда `asyncio.run()` завершается, вы возвращаетесь обратно **в синхронный мир**, к следующим строкам вашего скрипта (если они есть).

Ключевой вывод: `asyncio.run()` — это ваш главный инструмент для запуска асинхронной логики. Вы пишете всю асинхронную часть программы внутри корутин, а затем запускаете самую главную из них с помощью одного простого вызова.

До сих пор мы использовали `await` для последовательного выполнения задач. Наша программа дожидалась завершения одной корутины, прежде чем перейти к следующей. Это

было похоже на нашего повара, который готовил блюда строго по очереди.

Но вся мощь асинхронности раскрывается тогда, когда мы можем запустить несколько "долгих" операций и не ждать их, а заниматься в это время другими делами. Как заставить повара поставить воду для супа на плиту и, не дожидаясь закипания, сразу же начать резать овощи?

Для этого нам нужно научиться различать два способа вызова корутины.

Способ 1: `await my_coro()` (Сделать и дождаться)

Это то, что мы уже знаем.

- **Аналогия:** Повар решает сварить соус. Он берет ингредиенты, ставит их на плиту и **стоит над ними, помешивая, пока соус не будет готов**. Он полностью сфокусирован на этой задаче и не может делать ничего другого.
- **Что происходит в коде:** Программа доходит до этой строки, запускает корутину `my_coro` и **приостанавливается**, ожидая ее полного завершения. Она не пойдет дальше, пока не получит результат. Это **последовательное** выполнение.

Способ 2: `asyncio.create_task(my_coro())` (Поставить задачу и уйти)

Это новый и очень мощный инструмент.

- **Аналогия:** Повар ставит большую кастрюлю с водой на плиту, чтобы она закипела для пасты. Он включает конфорку и **сразу же отходит**, чтобы заняться нарезкой овощей для салата. Он не ждет закипания. Он просто дал команду "начать кипячение" и сразу переключился на другую работу. Задача кипячения воды теперь выполняется в "фоновом режиме".
- **Что происходит в коде:** Эта функция говорит Циклу событий: "Вот корутина `my_coro`. Пожалуйста, создай для нее **Задачу** (Task) и начни выполнять ее в фоновом режиме, как только сможешь". Сама программа **не ждет** и **немедленно** переходит к следующей строке кода. Функция возвращает объект Task, который является "ручкой" или ссылкой

на эту фоновую операцию.

Сравнение в одной таблице

Характеристика	<code>await my_coro()</code>	<code>task = asyncio.create_task(my_coro())</code>
Действие	Запустить и дождаться завершения.	Запланировать запуск в "фоне".
Поведение	Блокирует выполнение текущей корутины до завершения <code>my_coro</code> .	Не блокирует. Сразу переходит к следующей строке.
Возвращаемое значение	Результат выполнения <code>my_coro</code> .	Объект <code>Task</code> , который управляет корутиной.

Ключевой вывод: `asyncio.create_task()` — это наш способ сказать: "Начни делать это, а я пока займусь другими делами". Это основа для построения конкурентных, по-настоящему быстрых I/O-bound приложений.

В теории мы узнали, что `asyncio.create_task()` планирует запуск корутины в "фоновом режиме" и не ждет ее завершения. Давайте посмотрим, что произойдет, если мы создадим несколько таких "фоновых" задач и больше ничего не сделаем.

Мы напишем программу, которая запускает две медленные задачи. Каждая задача ждет несколько секунд и печатает сообщение.

1. Блок "Пример кода" (Замените старый код на этот)

Мы добавим `await asyncio.sleep(0.1)` в конце `main`. Без этого `main` завершится настолько

быстро, что планировщик (Event Loop) может даже не успеть запустить наши фоновые задачи.



```
import asyncio

async def slow_task(name, delay):
    # Эта строка выполнится, только если main даст управление циклу событий
    print(f"Задача '{name}' запущена, будет выполняться {delay} сек.")

    await asyncio.sleep(delay)

    # А вот сюда мы точно не дойдем!
    print(f"Задача '{name}' завершена.")

async def main():
    print("Главная корутина 'main' начала работу.")

    # 1. Планируем выполнение задач (ставим в очередь)
    # create_task работает мгновенно и не ждет старта задачи
    asyncio.create_task(slow_task("Быстрая", 1))
    asyncio.create_task(slow_task("Медленная", 2))

    print("Главная корутина 'main' завершила свою работу.")

    # 2. ВАЖНЫЙ МОМЕНТ:
    # Мы делаем крошечную паузу, чтобы передать управление Event Loop.
    # Это позволит задачам хотя бы НАЧАТЬСЯ (вывести первый print).
    # Если убрать эту строку, программа может завершиться мгновенно.
    await asyncio.sleep(0.1)

# Запускаем нашу главную корутину
asyncio.run(main())

print("Программа полностью завершена.")
```

Блок "Что вы увидите в консоли?" (Замените вывод)

Обратите внимание на порядок: сначала main прощается, потом задачи стартуют, и только в самом конце программа завершается глобально.

Главная корутина 'main' начала работу.
Главная корутина 'main' завершила свою работу.
Задача 'Быстрая' запущена, будет выполняться 1 сек.
Задача 'Медленная' запущена, будет выполняться 2 сек.
Программа полностью завершена.

(Примечание: порядок строк запуска задач "Быстрая"/"Медленная" может меняться, это нормально).

Блок "Разбор полетов: в чем проблема?" (Замените старое объяснение)

Самое главное — мы не увидели сообщений о **завершении** задач ("Задача завершена"). Почему так произошло, ведь мы их запустили?

Давайте проследим за логикой работы `asyncio.run()`:

1. **Запуск:** `main` мгновенно создает две задачи через `create_task`. Они попадают в очередь планировщика, но еще не работают.
2. **Пауза:** `main` доходит до `await asyncio.sleep(0.1)`. Слово `await` сигнализирует: "Я пока занят ожиданием, Цикл событий, можешь переключиться на другие дела".
3. **Переключение контекста:** Цикл событий видит свободное время и запускает наши фоновые задачи.
 - Они успевают напечатать "Задача запущена...".
 - Затем они доходят до своих длинных `await asyncio.sleep(1)` и тоже засыпают.
4. **Финал:** Проходит 0.1 секунды. `main` просыпается и завершает свою работу.
5. **Очистка:** Функция `asyncio.run()` видит, что главная корутина (`main`) закончилась. Это сигнал к остановке. `asyncio.run()` **принудительно отменяет (cancels)** все задачи, которые еще не успели закончиться.

Аналогия: Вы поставили блюда в духовку (создали задачи) и ушли смотреть короткую рекламу (`sleep 0.1`). Пока шла реклама, духовка начала греться. Но как только реклама кончилась, вы выключили рубильник во всем доме (`main` завершилась). Естественно, блюда не успели приготовиться до конца.

Ключевой вывод: `asyncio.create_task()` — это только старт. Если вы хотите, чтобы задачи гарантированно выполнились до конца, вы должны явно попросить программу **дождаться**

их. Как это сделать — мы узнаем на следующем шаге.

В предыдущем шаге мы столкнулись с проблемой: наши фоновые задачи были "убиты", потому что главная корутина `main` завершилась раньше них. Как заставить `main` дожидаться их?

Решение простое и элегантное. Мы должны использовать уже знакомое нам ключевое слово `await`.

Объект `Task` является "ожидаемым" (`awaitable`)

Помните, мы говорили, что `asyncio.create_task()` возвращает объект `Task`? Этот объект — не просто "пустышка". Он является "**awaitable**" (ожидаемым), точно так же, как и сама корутина.

Это значит, что на нем можно вызвать `await`.

Когда вы пишете `await my_task`, вы говорите Циклу событий: "**Приостанови мою работу здесь (внутри `main`), пока задача, на которую указывает `my_task`, не будет помечена как выполненная**".

Как это работает на практике?

План действий для корректного управления фоновой задачей теперь состоит из двух шагов:

1. **Запустить и сохранить:** Вызвать `asyncio.create_task()` и **обязательно сохранить** возвращенный объект `Task` в переменную.

codePython

```
task_1 = asyncio.create_task(slow_task("Задача 1", 3))
```

2. **Дождаться в конце:** В том месте, где вам нужно гарантировать завершение задачи (обычно в конце главной корутины), вы используете `await` для этой переменной.

codePython

```
await task_1
```

Аналогия с духовкой 2.0

Давайте вернемся к нашей аналогии с приготовлением ужина.

1. `pie_task = asyncio.create_task(bake_pie())` — это вы **ставите пирог в духовку и включаете таймер**. Процесс пошел, но вы свободны. Переменная `pie_task` — это ваш "таймер", который отслеживает процесс.
2. Вы идете делать салат, накрывать на стол (выполняете другие строки кода в `main`).
3. В конце, перед тем как звать всех к столу, вы должны убедиться, что пирог готов. Вы смотрите на таймер и ждете, пока он прозвенит. Это и есть `await pie_task`. Вы **ждете сигнала от таймера**, что фоновая задача завершена.

Только после того, как таймер прозвенел, вы можете с уверенностью сказать: "Ужин готов!" и завершать программу.

Ключевой вывод: `asyncio.create_task()` — это команда "Начать!", а `await` для соответствующего объекта `Task` — это команда "Дождаться окончания!". Оба шага необходимы для корректного управления фоновыми задачами.

Группировка задач с `asyncio.gather()`

В прошлом уроке мы научились запускать фоновые задачи и дожидаться их завершения по одной:

codePython

```
task1 = asyncio.create_task(some_task())
task2 = asyncio.create_task(another_task())

# ... какая-то другая работа ...

await task1

await task2
```

Это отлично работает для двух-трех задач. Но представьте, что вам нужно запустить 10 или 100 задач одновременно (например, скачать 100 веб-страниц). Писать и ожидать каждую задачу по отдельности станет очень громоздко и неудобно.

Именно для этой цели в `asyncio` существует мощный и элегантный инструмент — `asyncio.gather()`.

*`asyncio.gather(*awaitables)` — это функция, которая принимает одну или несколько "ожидаемых" сущностей (корутин, задач), запускает их конкурентно и возвращает результат только тогда, когда они **все** завершатся.*

Аналогия: Старт забега

Представьте себе тренера на стадионе, которому нужно провести забег с тремя спортсменами.

- **Неэффективный способ (по одной задаче):** Тренер подходит к первому спортсмену, говорит "Беги!", бежит рядом с ним до финиша, дожидается его. Потом возвращается на старт ко второму, говорит "Беги!", и снова ждет его на финише. Это абсурд.
- **Эффективный способ (с `gather`):** Тренер собирает всех спортсменов на стартовой линии (`gather` принимает всех "участников"). Он дает один стартовый сигнал (вызов `await asyncio.gather(...)`), и **все спортсмены начинают бежать одновременно** (конкурентно). Тренер идет на финишную черту и ждет, пока **последний** из них не пересечет ее.

В чем главные преимущества `gather`?

1. **Краткость:** Вместо множества строк с `create_task` и `await task` у вас будет всего одна.
2. **Удобство:** `gather` достаточно умен, чтобы самостоятельно "обернуть" переданные ему корутины в Задачи. Вам больше не нужно делать `asyncio.create_task()` вручную.
3. **Читаемость:** Код `await asyncio.gather(coro1, coro2, coro3)` гораздо яснее говорит о вашем намерении: "Запустить все это вместе и дождаться, пока все закончится".

Ключевой вывод: `asyncio.gather` — это ваш основной инструмент, когда нужно запустить "пачку" асинхронных операций одновременно и дождаться их полного завершения. Он делает

код чище, короче и понятнее.

Теперь давайте применим `asyncio.gather()` на практике. Мы создадим программу-симулятор кухни, где шеф-повар должен приготовить несколько блюд одновременно. Каждое блюдо — это корутина с разным временем "готовки".

Пример кода

Сравните этот код с тем, что мы писали в прошлом уроке. Обратите внимание, насколько он стал короче и чище.

```
import asyncio
import time

async def make_dish(name, cook_time):
    """Корутина, имитирующая приготовление блюда."""
    print(f"Начинаем готовить '{name}' (время: {cook_time} сек)...")
    await asyncio.sleep(cook_time)
    print(f"Блюдо '{name}' готово!")

async def main():
    print("Шеф-повар начинает одновременную готовку.")
    start_time = time.time()

    # Собираем все корутины и запускаем их конкурентно с помощью gather
    await asyncio.gather(
        make_dish("Паста", 3),
        make_dish("Салат", 1),
        make_dish("Хлебная корзина", 0.5)
    )

    end_time = time.time()
    print(f"\nВсе блюда готовы! Общее время: {end_time - start_time:.2f} секунд.")

# Запускаем программу
asyncio.run(main())
```

Что вы увидите в консоли?

Вывод будет примерно таким (порядок сообщений о готовности будет зависеть от времени):

codeCode

```
Шеф-повар начинает одновременную готовку.  
Начинаем готовить 'Паста' (время: 3 сек)...  
Начинаем готовить 'Салат' (время: 1 сек)...  
Начинаем готовить 'Хлебная корзина' (время: 0.5 сек)...  
Блюдо 'Хлебная корзина' готово!  
Блюдо 'Салат' готово!  
Блюдо 'Паста' готово!  
  
Все блюда готовы! Общее время: 3.00 секунд.
```

Разбор полетов

1. **Мгновенный старт:** Все три сообщения "Начинаем готовить..." появляются почти одновременно. Это доказывает, что `gather` немедленно запланировал все три корутины.
2. **Конкурентное выполнение:** Блюда "готовятся" (находятся в состоянии `await asyncio.sleep()`) параллельно.
3. **Время выполнения — ключ к пониманию:** Суммарное время готовки составляет $3 + 1 + 0.5 = 4.5$ секунды. Однако вся программа выполнилась примерно за **3 секунды**. Почему? Потому что общее время равно времени выполнения **самой долгой** задачи. Пока готовилась паста (3 сек), салат и хлеб успели приготовиться "внутри" этого же промежутка времени.
4. **Простота:** Нам не пришлось создавать переменные для каждой задачи и ждать их по отдельности. `gather` сделал всю грязную работу за нас.

Это и есть магия конкурентности. Мы не избавились от ожидания, но мы сделали так, чтобы несколько ожиданий происходили одновременно.

Мы увидели, что `asyncio.gather()` отлично подходит для запуска "процедур" — корутин, которые что-то делают, но ничего не возвращают (кроме `None` по умолчанию).

Но что, если наши корутины должны не просто напечатать сообщение, а **вернуть результат**? Например, одна корутина скачивает данные о пользователе, а другая — его последние транзакции. Нам нужно получить эти данные для дальнейшей обработки.

К счастью, `gather` спроектирован именно для этого.

asyncio.gather() не просто ждет завершения всех корутин, но и **собирает их результаты** в единый список.

Как это работает?

Внутри асинхронной функции (`async def`) вы можете использовать `return`, точно так же, как в обычной функции, чтобы вернуть значение.

Когда вы передаете несколько таких корутин в `gather`, происходит следующее:

1. `gather` запускает все корутины конкурентно.
2. Каждая корутина выполняется и, дойдя до `return`, завершается, "отдавая" свое возвращаемое значение.
3. `gather` "складирует" эти результаты у себя.
4. Когда **самая последняя** корутина завершается, `gather` отдает управление обратно, возвращая **список**, содержащий результаты всех корутин.

Важнейший момент: порядок результатов

Может показаться, что результаты в списке будут в том порядке, в котором корутины завершились. **Это не так!**

Результаты в списке, возвращаемом `gather`, всегда идут в том же порядке, в котором вы передали сами корутины в `gather`.

Давайте посмотрим на пример:

```
import asyncio

async def slow_task(): 1 usage new *
    await asyncio.sleep(2)
    return "Медленный"

async def fast_task(): 1 usage new *
    await asyncio.sleep(1)
    return "Быстрый"

async def main(): 1 usage new *
    results = await asyncio.gather(
        slow_task(), # Передаем медленную первой
        fast_task()  # Передаем быструю второй
    )
    print(results)

asyncio.run(main())
```

Несмотря на то, что `fast_task` завершится раньше, вывод будет строго таким:

```
['Медленный', 'Быстрый']
```

Это очень удобно, потому что вы всегда знаете, какому вызову соответствует какой результат.

Ключевой вывод: `await asyncio.gather(...)` — это не только способ дождаться, но и основной способ получить результаты от группы конкурентно выполняющихся корутин. Результатом будет список, порядок элементов в котором соответствует порядку переданных корутин.

Структурная конкурентность с `TaskGroup` (Python 3.11+)

Мы только что изучили `asyncio.gather()`. Это классический и удобный способ запустить пачку задач и получить список результатов.

Но начиная с **Python 3.11**, в языке появился новый инструмент — `asyncio.TaskGroup`. Он реализует концепцию, которая называется **Structured Concurrency** (Структурная конкурентность).

В чем проблема `gather`?

Когда вы запускаете `gather`, ваши задачи отправляются "в свободное плавание". Если одна из них упадет с ошибкой, остальные могут продолжить работать вхолостую, или (что хуже) зависнуть, и вы не будете знать, что с ними делать. `gather` — это немного хаос.

TaskGroup — это порядок

TaskGroup работает как контекстный менеджер (with). Главное правило: **программа не выйдет из блока with, пока ВСЕ задачи внутри него не завершатся.**

Это создает "безопасную зону": вы точно знаете, что после строчки с `async with` все ваши асинхронные операции либо успешно завершились, либо были корректно остановлены.

Как получить результаты?

Здесь подход отличается от `gather`. TaskGroup сам по себе **не возвращает список результатов.**

Вместо этого мы:

1. Создаем задачи внутри группы и **сохраняем их в переменные.**
2. Ждем выхода из блока `async with`.
3. Забираем результаты из переменных через метод `.result()`.

Давайте перепишем наш пример на новый лад:

Главная фишка: Безопасность при ошибках

Самое крутое в TaskGroup — это то, как он реагирует на проблемы.

Если **одна** из задач внутри группы упадет с ошибкой:

1. TaskGroup моментально **отменит** (cancel) все остальные работающие задачи в этой группе.
2. Он дождется их остановки.
3. И только потом выбросит исключение (специальный `ExceptionGroup`), сообщающее о проблеме.

Это спасает от "зомби-процессов", когда скрипт упал, а какие-то фоновые скачивания продолжают висеть в памяти.

Сравнение подходов

Используйте `asyncio.gather`, если:

- Вам нужно просто быстро "сбегать" за данными в 10 мест одновременно.

- Результаты однотипны (например, список строк с 10 сайтов).
- Вам нужен простой список результатов на выходе: `results = await gather(...)`.

Используйте `asyncio.TaskGroup`, если:

- Вы пишете сложную логику приложения.
- Задачи связаны между собой (нет смысла качать транзакции, если скачивание пользователя упало).
- Вам важна надежность и правильная очистка ресурсов при ошибках.

Резюме

`TaskGroup` — это современный, более строгий и безопасный брат `gather`. Он требует чуть больше кода (нужно сохранять задачи в переменные), но взамен дает полный контроль над тем, что происходит в вашей асинхронной программе. Если вы пользуетесь Python 3.11+, старайтесь привыкать именно к нему.

Проблема "Гонки данных" (Race Condition)

В предыдущем модуле мы научились запускать множество задач одновременно и радовались, как быстро теперь выполняются наши программы. Но такая конкурентная работа с общими ресурсами таит в себе скрытые опасности.

Одна из самых известных и коварных проблем в конкурентном программировании — это «гонка данных» (**Race Condition**).

Гонка данных возникает тогда, когда несколько задач пытаются изменить одни и те же данные, и итоговый результат зависит от того, в каком порядке задачи получают доступ к этим данным.

В реальном приложении обновление данных часто выглядит так:

1. **Прочитать** текущее значение (например, запрос к базе данных).
2. **Подождать** (сетевая задержка).

3. **Вычислить** новое значение.
4. **Сохранить** новое значение.

Давайте смоделируем эту ситуацию в коде:

```
import asyncio

COUNTER = 0

async def increment():
    global COUNTER

    # Шаг 1: Читаем текущее значение
    # (как будто сделали SELECT из базы данных)
    temp_value = COUNTER

    # Шаг 2: Имитируем бурную деятельность или ожидание сети.
    # ВНИМАНИЕ: Именно здесь управление передается другим задачам!
    await asyncio.sleep(0.1)

    # Шаг 3: Увеличиваем и записываем обратно
    COUNTER = temp_value + 1

async def main():
    # Запускаем 10 задач конкурентно
    await asyncio.gather(*(increment() for _ in range(10)))

asyncio.run(main())
print(f"Итоговый счетчик: {COUNTER}")
```

Вопрос: каким будет значение COUNTER в конце?

Ожидание: 10.

Реальность: Скорее всего, 1.

Почему так произошло?

Может показаться, что раз Python (с asyncio) работает в одном потоке, то всё должно быть безопасно. Но это не так.

Проблема кроется в **переключении контекста**. Ключевое слово `await` сигнализирует циклу событий: "Я пока занят ожиданием, можешь переключиться на другую задачу".

Вот как выглядит "плохой" сценарий для нашего кода:

1. **Задача А** начинает работу. Она читает COUNTER (равен 0) в переменную `temp_value`.
2. **Задача А** встречает `await`. Она "засыпает" и уступает место.
3. Цикл событий запускает **Задачу Б**.
4. **Задача Б** читает COUNTER. Поскольку Задача А **еще не успела** сохранить новое значение, COUNTER всё еще равен **0**!
5. Задача Б тоже уходит в `await`.
6. ...Представьте, что так сделали все 10 задач. У всех в кармане `temp_value = 0`.
7. Спустя 0.1 секунды задачи начинают просыпаться.
8. **Задача А** просыпается, делает $0 + 1$ и записывает **1** в COUNTER.
9. **Задача Б** просыпается, делает $0 + 1$ (она ведь запомнила 0) и... **перезаписывает** COUNTER той же самой единицей.

Итог: Мы потеряли 9 инкрементов из 10.

Важное отличие от многопоточности (threading)

Если вы пришли из мира многопоточности, вы знаете, что там даже операция `COUNTER += 1` опасна.

В `asyncio` всё немного проще: операция `COUNTER += 1` (без `await` внутри) **безопасна**, потому что `asyncio` не прерывает выполнение кода посреди строки.

Гонка данных в `asyncio` возникает **только** тогда, когда цепочка действий "чтение — изменение — запись" разрывается ключевым словом `await`.

Что такое "Критическая секция"?

Участок кода, который работает с общим ресурсом и не должен быть прерван другими задачами, называется **критической секцией**.

В нашем примере критическая секция — это весь процесс от чтения `temp_value` до обновления COUNTER.

Чтобы исправить ошибку и получить в итоге число 10, нам нужно гарантировать, что пока одна задача находится внутри критической секции, другие задачи будут **ждать** своей очереди и не полезут читать COUNTER раньше времени.

Для этого в asyncio существуют примитивы синхронизации, и самый главный из них — **Lock (Замок)**. О нем мы поговорим в следующем уроке.

В теории мы разобрали, как две задачи могут "помешать" друг другу при обновлении общего счетчика. Давайте напишем программу, которая запускает не две, а сразу 10 000 задач, каждая из которых попытается увеличить счетчик на единицу.

Если бы все работало правильно, итоговый результат должен быть 10 000. Но так ли это будет?

Пример кода

Чтобы сделать проблему более явной и почти гарантированно воспроизводимой, мы немного изменим операцию +=. Мы вручную разделим ее на три шага (чтение, увеличение, запись) и добавим между ними `await asyncio.sleep(0)`. Эта команда — специальный трюк: она говорит циклу событий: "Я готов уступить управление прямо сейчас, переключись на другую задачу, если можешь". Это идеальный момент для возникновения гонки.

```
# Наш общий ресурс, который будут изменять все задачи
COUNTER = 0
# Сколько задач мы хотим запустить
TASKS_COUNT = 10_000

async def increment():
    """
    Корутина, которая инкрементирует общий счетчик
    небезопасным (неатомарным) способом.
    """
    global COUNTER

    # 1. Читаем текущее значение
    temp_counter = COUNTER

    # В этой точке мы добровольно отдаем управление.
    # Другая задача может начать выполняться и тоже
    # прочитать СТАРОЕ значение COUNTER.
    await asyncio.sleep(0)

    # 2. Увеличиваем локальное значение
    temp_counter += 1

    # 3. Записываем новое значение обратно
    COUNTER = temp_counter

async def main():
    print(f"Ожидаемое значение счетчика: {TASKS_COUNT}")

    # Запускаем 10 000 задач конкурентно
    tasks = [increment() for _ in range(TASKS_COUNT)]
    await asyncio.gather(*tasks)

    print(f"Итоговое значение счетчика: {COUNTER}")

# Запускаем программу
asyncio.run(main())
```

Что вы увидите в консоли?

Запустите код несколько раз. Вы увидите, что результат всегда разный, и он **никогда** не будет равен 10 000.

Пример 1-го запуска:

codeCode

Ожидаемое значение счетчика: 10000

Итоговое значение счетчика: 1

Пример 2-го запуска:

codeCode

Ожидаемое значение счетчика: 10000

Итоговое значение счетчика: 2

(У вас могут быть и другие значения, но всегда очень маленькие)

Разбор полетов

Этот результат наглядно демонстрирует хаос, который вносит гонка данных. Что произошло?

1. Почти все 10 000 задач были запущены одновременно.
2. Каждая из них успела выполнить первую строку `temp_counter = COUNTER`, прочитав исходное значение `COUNTER` (равное **0**).
3. Затем они все ушли в состояние `await asyncio.sleep(0)`, уступив управление.
4. Цикл событий по очереди "пробуждал" их. Каждая проснувшаяся задача брала свое локальное значение `temp_counter` (которое равно **0**), прибавляла к нему 1, получала 1 и записывала эту 1 в глобальную переменную `COUNTER`.
5. В итоге последняя завершившаяся задача записала в `COUNTER` значение 1, перезаписав результаты всех предыдущих.

Мы "потеряли" 9999 операций инкремента. Этот пример показывает, насколько критично

защищать участки кода, которые работают с общими изменяемыми данными.

В предыдущем шаге мы увидели, как конкурентный доступ к общему счетчику привел к хаосу и неверному результату. Нам нужен механизм, который бы заставил задачи вести себя "вежливо" и работать с общим ресурсом по очереди.

Для решения этой проблемы в `asyncio` существует примитив синхронизации под названием **`asyncio.Lock`**.

`asyncio.Lock` (блокировка) — это объект, который может находиться в двух состояниях:

«захвачен» (`locked`) или «свободен» (`unlocked`). Он работает как пропуск: только та задача, которая владеет пропуском, может войти в защищенную зону

Аналогия: Ключ от комнаты

Лучшая аналогия для `Lock` — это ключ от одноместной комнаты (например, примерочной или ванной).

- Есть одна комната (наш **общий ресурс**, переменная `COUNTER`).
- Есть один ключ (`Lock`).
- Есть много людей (наши **задачи**), которые хотят попасть в комнату.

Правила просты:

1. Чтобы войти в комнату, нужно взять ключ. Если ключ на месте (`lock` "свободен"), ты его берешь и заходишь, заперев за собой дверь (`lock` становится "захвачен").
2. Если тыходишь, а ключа нет (`lock` "захвачен"), ты не можешь войти. Ты должен **встать в очередь и ждать**, пока тот, кто внутри, не выйдет и не вернет ключ.
3. Выйдя из комнаты, ты **обязан** вернуть ключ на место (освободить `lock`), чтобы следующий человек из очереди мог его взять.

`asyncio.Lock` гарантирует, что только одна корутина может «владеть» блокировкой в любой момент времени.

Критическая секция и синтаксис `async with`

Участок кода, который мы защищаем с помощью блокировки (в нашем случае, это чтение, изменение и запись счетчика), называется **критической секцией**.

Как же использовать Lock в коде? Самый правильный, безопасный и современный способ — это использовать конструкцию `async with`.

Вот как это выглядит:

Что делает `async with lock`?

1. Перед тем как войти в этот блок, задача пытается **захватить** блокировку (`await lock.acquire()`).
2. Если блокировка свободна, задача захватывает ее и начинает выполнять код внутри блока.
3. Если блокировка занята другой задачей, текущая задача "засыпает" и ждет, пока блокировка не освободится.
4. Самое главное: когда выполнение выходит из блока `async with` (неважно, завершился он успешно или произошла ошибка), блокировка **автоматически освобождается** (`lock.release()`). Это гарантирует, что ключ никогда не будет "потерян".

Ключевой вывод: `asyncio.Lock` с помощью `async with` — это инструмент, который превращает хаотичную гонку в упорядоченную очередь, гарантируя, что критическая секция кода выполняется только одной задачей за раз.

4.2 Ограничение ресурсов с `asyncio.Semaphore`

В предыдущем уроке мы решали проблему "гонки данных", когда множество задач пытались одновременно изменить один ресурс. Мы использовали `asyncio.Lock`, чтобы в "критическую секцию" могла войти только **одна** задача за раз. Это было похоже на ключ от одноместной комнаты.

Но что, если нам не нужна такая строгая эксклюзивность? Что, если мы хотим, чтобы в "комнату" могло войти, скажем, 5 или 10 задач одновременно, но не больше?

Когда это нужно?

Существует множество ситуаций, когда нам нужно ограничить "параллелизм" — количество одновременно выполняемых операций, — не сводя его к единице.

Самый частый пример: работа с внешними API.

Представьте, вы работаете с API какого-нибудь сервиса (например, для получения прогноза погоды или курсов валют). В документации этого сервиса почти всегда есть раздел "Rate Limiting" (Ограничение частоты запросов), где написано что-то вроде:

*Вы можете делать не более **10 запросов в секунду** с вашего IP-адреса.*

Что произойдет, если вы напишете код, который должен получить данные для 1000 городов, и запустите его с помощью `await asyncio.gather(...)`?

Ваша программа почти мгновенно попытается отправить 1000 запросов к API. Сервис увидит этот шквал запросов, поймет, что вы превысили лимит, и вернет вам ошибку 429 Too Many Requests на большинство из них. В худшем случае он может временно заблокировать ваш IP-адрес.

Другие примеры:

- **Ограничения базы данных:** Пул соединений к базе данных часто имеет ограниченный размер. Вы не можете открыть 1000 соединений одновременно.
- **Защита от перегрузки:** Вы не хотите перегружать запросами свой собственный или чужой сервис.
- **Управление локальными ресурсами:** Например, ограничение количества одновременных операций чтения/записи на диск.

Аналогия: Ночной клуб с ограниченной вместимостью

Представьте себе популярный ночной клуб, внутри которого может находиться не более **10 человек** одновременно (это его "пропускная способность").

- На входе стоит строгий охранник ("баунсер").
- Первые 10 человек, которые приходят, сразу проходят внутрь.
- Когда приходит 11-й человек, охранник вежливо просит его подождать в очереди.

- Как только один человек выходит из клуба, охранник тут же пропускает первого из очереди.

Таким образом, внутри клуба всегда находится не более 10 человек, хотя в очереди на вход может стоять гораздо больше.

В нашем коде нам нужен такой же "баунсер", который будет пропускать к API не более 10 задач одновременно, а остальные будут вежливо просить подождать в очереди.

Для решения именно таких задач в `asyncio` существует другой примитив синхронизации — **Семафор**.

Ключевой вывод: Если `Lock` нужен для **эксклюзивного** доступа (разрешен только **один**), то Семафор нужен для **ограниченного** доступа (разрешено **N**). Давайте напишем программу, которая имитирует ситуацию, описанную в предыдущем шаге. Мы хотим обработать 100 элементов (например, получить данные для 100 пользователей), и для этого запускаем 100 задач конкурентно с помощью `asyncio.gather()`.

Каждая задача будет имитировать сетевой запрос: она немного "подождет", как будто ждет ответа от сервера.

Пример кода

Этот код не использует никаких ограничителей. Он просто запускает все, что мы ему даем, так быстро, как только может.

```
import asyncio
import time

# Количество "запросов", которые мы хотим сделать
TOTAL_REQUESTS = 100

async def make_request(request_num):
    """Имитирует выполнение сетевого запроса."""
    print(f"[{time.time():.2f}] Начинаю запрос #{request_num}...")
    # Имитация ожидания ответа от сети
    await asyncio.sleep(1)
    print(f"[{time.time():.2f}] Запрос #{request_num} завершен.")

async def main():
    print(f"Запускаем {TOTAL_REQUESTS} запросов одновременно...")
    start_time = time.time()

    # Создаем и запускаем все 100 задач сразу
    tasks = [make_request(i) for i in range(TOTAL_REQUESTS)]
    await asyncio.gather(*tasks)

    end_time = time.time()
    print(f"\nВсе {TOTAL_REQUESTS} запросов выполнены за {end_time -
start_time:.2f} сек.")

asyncio.run(main())
```

Запускаем 100 запросов одновременно...

[1666598400.10] Начинаю **запрос #0...**

[1666598400.10] Начинаю **запрос #1...**

[1666598400.10] Начинаю **запрос #2...**

...

[1666598400.11] Начинаю **запрос #98...**

[1666598400.11] Начинаю **запрос #99...**

(далее, через секунду, пойдут сообщения о завершении)

[1666598401.11] **Запрос #0** завершен.

[1666598401.11] **Запрос #1** завершен.

...

(ваши временные метки будут другими)

Разбор полетов: в чем проблема?

- **Мгновенный старт:** Обратите внимание на временные метки. Все 100 сообщений "Начинаю запрос..." появились в течение сотых долей секунды. Это значит, что наша программа **попыталась инициировать 100 сетевых соединений практически одновременно.**
- **Реальный мир:** Теперь представьте, что `asyncio.sleep(1)` — это настоящий запрос к API с лимитом "10 запросов в секунду". Наш код отправил бы 100 запросов за ~0.01 секунды. Первые 10, возможно, были бы приняты, а остальные 90 вернулись бы с ошибкой.

Этот код демонстрирует "наивный" подход к конкурентности. Он эффективен, если у вас нет внешних ограничений. Но в реальном мире такие ограничения есть почти всегда.

Нам нужен способ запустить все 100 задач, но сделать так, чтобы в состоянии "активного запроса" находилось не более N из них в любой момент времени.

Именно эту задачу решает `asyncio.Semaphore`, с которым мы познакомимся на следующем шаге.

В предыдущем шаге мы увидели, как наша программа попыталась выполнить 100 операций одновременно, что в реальном мире привело бы к ошибкам или блокировке. Нам нужен "регулирующий", который будет контролировать этот поток.

Для решения этой задачи в `asyncio` и существует `asyncio.Semaphore`.

Семафор (*Semaphore*) — это примитив синхронизации, который позволяет ограничить доступ к определенному ресурсу, пропуская не более N задач одновременно. По своей сути, это продвинутый счетчик.

Аналогия: Парковка на N мест

Лучшая аналогия для `asyncio.Semaphore(N)` — это парковка с ограниченным количеством мест.

Представьте парковку, на которой ровно **10 мест** ($N=10$). На въезде стоит шлагбаум с электронным табло, которое показывает количество свободных мест.

- **Машины** — это наши **задачи** (корутины), которые хотят выполнить операцию (например, сделать API-запрос).
- **Парковочные места** — это "**разрешения**" на выполнение, которыми управляет семафор.
- **Шлагбаум с табло** — это сам **семафор**.

Вот как все работает:

1. **Парковка пуста (10 свободных мест)**. Первые 10 машин (задач), которые подъезжают, сразу же проезжают через шлагбаум и занимают места. На табло теперь 0 свободных мест.
2. **Парковка заполнена**. Когда подъезжает 11-я машина, она видит, что мест нет. Шлагбаум для нее закрыт. Она не уезжает, а **встает в очередь и ждет**. За ней выстраиваются 12-я, 13-я и так далее.

3. **Место освобождается.** Как только одна из первых 10 машин выезжает с парковки, на табло появляется 1 свободное место. Шлагбаум тут же открывается для первой машины в очереди, и она заезжает. На табло снова 0.

Таким образом, семафор гарантирует, что на "парковке" (внутри нашего защищенного кода) никогда не будет находиться более N машин (задач) одновременно.

Как это выглядит в коде?

Использование семафора очень похоже на использование Lock — через конструкцию `async with`.

1. **"Строим парковку":** Сначала мы создаем экземпляр семафора, указывая его "емкость".

```
# Создаем семафор, который пропустит не более 10 задач одновременно
semaphore = asyncio.Semaphore(10)
```

2. **"Проезжаем через шлагбаум":** Внутри нашей корутины мы оборачиваем "дорогу" операцией в `async with`.

```
async def make_request(request_num):
    async with semaphore:
        # --- НАЧАЛО ОГРАНИЧЕННОЙ ЗОНЫ ---
        # Этот код будет выполняться одновременно
        # не более чем для 10 задач.
        print(f"Запрос #{request_num} 'заехал на парковку'")
        await asyncio.sleep(1) # Имитация работы
        # --- КОНЕЦ ОГРАНИЧЕННОЙ ЗОНЫ ---
    print(f"Запрос #{request_num} 'выехал с парковки'")
```

Конструкция `async with semaphore`: автоматически "захватывает" одно из N разрешений перед входом в блок и "освобождает" его при выходе. Если все разрешения заняты, задача будет просто ждать своей очереди.

Ключевой вывод: `asyncio.Semaphore(N)` — это ваш «шлагбаум», который гарантирует, что в определенный участок кода одновременно войдет не более `N` задач. Это идеальный инструмент для работы с ресурсами, имеющими ограничение на частоту использования.

Обработка ошибок и отмена задач

До сих пор мы писали код, который, как мы надеялись, всегда будет работать успешно. Но в реальном мире все ломается: сети отключаются, серверы не отвечают, данные приходят в неверном формате. Надежная программа должна уметь грамотно обрабатывать такие сбои.

Вы уже наверняка знакомы со стандартным механизмом обработки ошибок в Python — конструкцией `try...except`.

```
# Синхронный пример
try:
    result = 10 / 0
except ZeroDivisionError:
    print("Ошибка: деление на ноль!")
```

Возникает логичный вопрос: а как эта конструкция ведет себя в асинхронном мире, где есть `await`? Не ломает ли асинхронная магия привычную обработку ошибок?

Представьте, что вы в ресторане (вы — корутина `main`). Вы заказываете экзотическое блюдо у официанта (вызываете `await risky_dish()`).

- `try:` — это ваше намерение: "Я попробую заказать это блюдо..."
- `await risky_dish()` — это момент, когда вы ждете, пока официант вернется с кухни.
- `except SomeError:` — это ваш план Б: "...но если официант вернется и скажет, что на кухне все сгорело, я просто закажу пиццу".

Когда официант возвращается (когда `await` завершается), он приносит либо блюдо (результат), либо плохие новости (исключение). Если он приносит плохие новости, ваш план Б (`except`) немедленно срабатывает.

Как это выглядит в коде?

```
import asyncio

# Корутина, которая всегда "ломается"
async def risky_operation():
    print("Начинаю рискованную операцию...")
    await asyncio.sleep(1)
    # Здесь происходит ошибка
    return 10 / 0

async def main():
    print("Попытка выполнить рискованную операцию.")
    try:
        # Оборачиваем await в try...except
        result = await risky_operation()
        print(f"Операция успешна, результат: {result}")
    except ZeroDivisionError:
        print("Поймал ошибку! Деление на ноль внутри корутины.")

    print("Программа продолжает работу.")

asyncio.run(main())
```

Ключевой вывод: Вы можете использовать try...except вокруг await так же, как вы использовали бы его вокруг обычного вызова функции. Если внутри risky_operation произойдет исключение, await обеспечит его "всплытие", и ваш except блок его поймает.

Мы напишем программу с двумя корутинами:

- `get_user_data()`: Эта корутина будет имитировать запрос к базе данных или API. В нашем примере она будет "падать", вызывая исключение `ValueError`, как будто пользователь с таким ID не найден.
- `main()`: Эта корутина будет вызывать `get_user_data()` и обрабатывать возможную ошибку с помощью try...except.

Пример кода

Скопируйте и запустите этот код.



```
import asyncio

async def get_user_data(user_id):
    """
    Имитирует запрос данных пользователя.
    Падает с ошибкой, если user_id равен 0.
    """
    print(f"Запрашиваю данные для пользователя {user_id}...")
    await asyncio.sleep(1) # Имитация сетевой задержки

    if user_id == 0:
        # Генерируем исключение, как будто пользователя не нашли
        raise ValueError(f"Пользователь с ID {user_id} не найден!")

    # Этот код не выполнится для user_id=0
    return {"id": user_id, "name": "Alice"}

async def main():
    print("--- Попытка 1: Запрашиваем существующего пользователя (ID=1) ---")
    try:
        user = await get_user_data(1)
        print(f"Успех! Данные получены: {user}")
    except ValueError as e:
        print(f"Произошла ошибка: {e}")

    print("\n" + "="*40 + "\n")

    print("--- Попытка 2: Запрашиваем несуществующего пользователя (ID=0) ---")
    try:
        user = await get_user_data(0)
        # Эта строка никогда не будет выполнена
        print(f"Успех! Данные получены: {user}")
    except ValueError as e:
        # Исключение, возникшее в get_user_data, будет поймано здесь
        print(f"Произошла ошибка, как и ожидалось: {e}")

    print("\nПрограмма корректно завершила работу.")

# Запускаем программу
asyncio.run(main())
```

```
--- Попытка 1: Запрашиваем существующего пользователя (ID=1) ---
Запрашиваю данные для пользователя 1...
(пауза 1 секунда)
Успех! Данные получены: {'id': 1, 'name': 'Alice'}

=====

--- Попытка 2: Запрашиваем несуществующего пользователя (ID=0) ---
Запрашиваю данные для пользователя 0...
(пауза 1 секунда)
Произошла ошибка, как и ожидалось: Пользователь с ID 0 не найден!

Программа корректно завершила работу.
```

Разбор полетов

1. **Первая попытка (успешная):** `get_user_data(1)` отработывает без ошибок, возвращает словарь, который `await` передает в переменную `user`, и выполняется код в блоке `try`.
2. **Вторая попытка (неудачная):**
 - Выполнение переходит в `get_user_data(0)`.
 - После `await asyncio.sleep(1)` корутина доходит до `raise ValueError(...)`.
 - В этот момент выполнение `get_user_data` прерывается, и объект исключения `ValueError` создается.
 - Оператор `await`, который "слушал" эту корутину в `main`, ловит это исключение.
 - `await` немедленно "перебрасывает" это исключение в том месте, где он был вызван — то есть, внутри блока `try`.
 - Выполнение блока `try` тут же прекращается, и Python ищет подходящий блок `except`. Он находит `except ValueError`, и выполняется код внутри этого блока.
3. **Программа не падает:** Обратите внимание, что после обработки исключения программа продолжила свою работу и напечатала финальное сообщение. Мы не получили красного текста с `Traceback`, потому что мы "поймали" ошибку.

Этот пример доказывает, что `await` действует как прозрачный "проводник" не только для результатов, но и для исключений.

Итак, `try...except` прекрасно работает с одним `await`. Но что произойдет, если мы запустим 10 задач с помощью `asyncio.gather()`, и одна из них, где-то в середине выполнения, "упадет" с ошибкой?

Поведет ли `gather` себя так же, как и обычный `await`? Ответ: да, но с одним очень важным нюансом.

Поведение `gather` по умолчанию: "Первая ошибка побеждает"

Стандартное поведение `asyncio.gather()` основано на принципе "fail-fast" (быстрый отказ).

*Если любая из задач, запущенных через `gather`, вызывает исключение, `gather` не ждет завершения остальных задач. Он немедленно **прекращает ожидание** и "пробрасывает" вверх исключение от **первой** сбойнувшей задачи.*

Аналогия: Командный забег

Представьте команду из трех бегунов, которые стартуют одновременно.

- `await asyncio.gather(run1, run2, run3)` — это вы, как тренер, ждете на финише, пока вся команда не закончит забег.
- Если во время забега бегун №2 спотыкается и получает травму (вызывает исключение), он по рации сообщает тренеру: "Я выбыл из строя!".
- Тренер (`gather`) не будет ждать, пока бегуны №1 и №3 добегут до финиша. Он немедленно фиксирует неудачу всей команды, поднимает красный флаг (пробрасывает исключение) и прекращает ожидание.

Важнейший нюанс: что происходит с остальными задачами?

И вот тут кроется главная опасность. Тренер поднял красный флаг, но бегуны №1 и №3 этого не знают! Они продолжают бежать, пока не добегут до финиша или пока судьи не закроют стадион (пока не завершится цикл событий).

Когда gather пробрасывает исключение, он не отменяет остальные запущенные задачи. Они продолжают выполняться в фоновом режиме "осиротевшими".

Это может привести к неприятным последствиям: утечкам ресурсов, неожиданным действиям, выполняющимся уже после того, как вы обработали ошибку.

Альтернативное поведение: `return_exceptions=True`

Что, если мы хотим дождаться всех, даже если некоторые потерпят неудачу? Например, мы скачиваем 100 файлов и хотим получить те, что скачались, даже если 5 из них не удалось загрузить.

Для этого у `gather` есть специальный параметр `return_exceptions=True`.

```
results = await asyncio.gather(  
    task1(),  
    task_that_fails(),  
    task3(),  
    return_exceptions=True # Включаем режим сбора ошибок  
)
```

Если установить этот флаг:

1. `gather` не будет пробрасывать исключение.
2. Он дождется завершения всех переданных задач, независимо от их успешности.
3. В итоговом списке `results` на месте успешных задач будут их результаты, а на месте "упавших" — объекты самих исключений.

Вы сможете потом пройтись по списку `results` и проверить, что является результатом, а что — ошибкой, с помощью `isinstance(item, Exception)`.

ключевой вывод:

- По умолчанию `gather` работает по принципу "все или ничего" и падает при первой же ошибке, оставляя остальные задачи работать в фоне.
- Для более гибкой обработки, когда вам нужны результаты успешных операций даже при наличии сбоев, используйте `asyncio.gather(..., return_exceptions=True)`.

В предыдущем шаге мы выяснили, что при сбое одной из задач в `gather`, остальные продолжают работать в фоновом режиме. Это почти всегда не то, чего мы хотим. Если главная цель (например, "обработать данные пользователя") провалилась, то связанные с ней подзадачи (скачать аватар, получить список друзей) уже не имеют смысла и лишь зря тратят ресурсы.

Нам нужен механизм, чтобы вежливо попросить эти фоновые задачи остановиться. Этот механизм — метод `.cancel()`.

`task.cancel()` — это метод объекта `Task`, который запрашивает отмену выполнения задачи.

Как на самом деле работает отмена?

Важно понимать: `task.cancel()` не работает как команда `kill` в операционной системе. Она не "убивает" задачу мгновенно. Вместо этого она использует элегантный асинхронный механизм:

- 1. Вы вызываете `my_task.cancel()`.*
- 2. Цикл событий планирует "вброс" специального исключения `asyncio.CancelledError` внутрь корутины, которой управляет `my_task`.*
- 3. Этот "вброс" произойдет не немедленно, а в тот момент, когда корутина дойдет до следующей точки `await`.*
- 4. Таким образом, задача приостанавливается на `await`, и вместо того, чтобы проснуться и продолжить работу, она просыпается от того, что в нее "прилетело" исключение `CancelledError`.*

`CancelledError`: это не ошибка, а сигнал

`asyncio.CancelledError` — это специальный тип исключения, который наследуется от `BaseException` (не от `Exception`). Он не означает, что в вашей программе ошибка. Он означает: "Внешний код попросил тебя остановиться".

Это сигнал к тому, что пора прекращать работу и прибираться за собой.

Аналогия: Пожарная тревога в офисе

Представьте, что ваша корутина — это офисный работник, который выполняет свою работу.

- `await` — это момент, когда работник ждет (например, пока загрузится страница или придет коллега).
- Вызов `task.cancel()` — это как будто кто-то включил пожарную тревогу.
- Когда тревога срабатывает (`CancelledError` выбрасывается), работник не продолжает печатать отчет. Его нормальная работа прерывается.
- Правильная реакция: работник сохраняет документы, выключает компьютер (это очистка ресурсов) и выходит из здания (позволяет исключению `CancelledError` "всплыть" дальше).
- Неправильная реакция: проигнорировать тревогу и продолжить работать.

Обработка отмены для очистки ресурсов

Хорошо написанная корутина должна быть готова к отмене. Если она работает с ресурсами, которые нужно закрывать (файлы, сетевые соединения), она должна перехватить `CancelledError`, выполнить очистку и затем позволить исключению распространиться дальше.

```
async def some_worker():
    resource = open_resource() # Открыли ресурс
    try:
        while True:
            # ... какая-то долгая работа с await ...
            await asyncio.sleep(1)
    finally:
        # Этот блок выполнится ВСЕГДА:
        # при нормальном выходе, при обычной ошибке И ПРИ ОТМЕНЕ.
        print("Очищаю ресурсы...")
        resource.close()
```

Или можно явно поймать `CancelledError` и перевыбросить его:

```
...
except asyncio.CancelledError:
    print("Меня отменили! Очищаю ресурсы...")
    resource.close()
    raise # Важно перевыбросить исключение, чтобы подтвердить
отмену
```

Ключевой вывод:

- `task.cancel()` — это вежливая просьба остановиться.
- Механизм отмены — это "вброс" исключения `CancelledError` в задачу в точке `await`.
- Правильно написанные корутины должны обрабатывать `CancelledError` (чаще всего в блоке `finally`) для корректной очистки ресурсов.

Тайм-ауты

Мы научились запускать задачи, управлять ими и даже обрабатывать ошибки. Мы использовали `await`, чтобы терпеливо дождаться результата. Но в этом терпении кроется скрытая опасность.

А что, если операция, которую мы ждем, **никогда не завершится?**

Проблема "зависших" операций

Представьте, что вы пишете код, который делает запрос к внешнему веб-серверу:

```
async def get_data_from_server():
    # ... код для установления соединения ...
    response = await server.read_response() # Ждем ответа

    return response
```

Что может пойти не так?

- Сервер может быть перегружен и очень медленно обрабатывать ваш запрос.
- Могут возникнуть проблемы с сетью, из-за которых ответ от сервера никогда не дойдет до вашей программы.
- Сервер может "зависнуть" сам и просто перестать отвечать, держа соединение открытым.

Во всех этих случаях ваша программа дойдет до строки `await server.read_response()` и будет ждать. И ждать. И ждать... **Бесконечно.**

Ваша корутина "зависнет" на этом `await`, и вся часть вашей программы, которая от нее зависит, тоже остановится. Для долгоживущих приложений (веб-серверы, боты, сервисы) это

катастрофа. Одна такая "зависшая" операция может со временем привести к исчерпанию ресурсов и полному отказу системы.

Аналогия: Бесконечное ожидание друга

Вы договорились встретиться с другом в кафе. Вы приходите и ждете. Проходит 10 минут, 20 минут, час... Друга все нет.

- **Наивная программа** будет сидеть и ждать вечно. Она никогда не покинет кафе.
- **Надежная, "умная" программа** поступит как разумный человек. Она заранее для себя решит: "Я подожду 15 минут, и если друг не появится, я уйду и буду заниматься своими делами".

Вот этот заранее установленный предел ожидания ("15 минут") и есть **тайм-аут**.

Тайм-ауты — это критически важный механизм защиты. Они позволяют нам установить "дедлайн" для любой асинхронной операции. Мы говорим программе:

*"Попробуй выполнить эту операцию. Я готов подождать, но **не более N секунд**. Если за это время результат не будет получен, прекращай ожидание и сообщи мне об ошибке".*

Это превращает потенциально бесконечное ожидание в контролируемую и обрабатываемую ошибку.

Ключевой вывод: Тайм-ауты — это не роскошь, а необходимость для создания стабильных и отказоустойчивых асинхронных приложений. Они защищают вашу программу от "зависания" из-за слишком медленных или не отвечающих внешних систем.

Мы определили проблему "зависших" операций и необходимость устанавливать "дедлайн" на их выполнение. Для ее решения в `asyncio` есть специальная, очень удобная функция — `asyncio.wait_for()`.

`asyncio.wait_for(awaitable, timeout)` — это функция-обертка, которая запускает "ожидаемую" (`awaitable`) операцию, но следит за временем. Она гарантирует, что ожидание не продлится дольше, чем указанный `timeout`.

Синтаксис

Использование `wait_for` очень интуитивно. Вы просто "оборачиваете" в нее ту корутину, выполнение которой хотите ограничить по времени:

```
try:
    # Ждем завершения some_coroutine(), но не более 5 секунд
    result = await asyncio.wait_for(some_coroutine(), timeout=5.0)
    print("Успех! Результат получен вовремя.")
except asyncio.TimeoutError:
    print("Провал! Операция не уложилась в 5 секунд.")
```

Важные компоненты:

- `some_coroutine()`: Любой "ожидаемый" объект (обычно корутина), который вы хотите запустить.
- `timeout=5.0`: Максимальное время ожидания в секундах. Может быть целым числом или с плавающей точкой.
- `await`: Сама функция `wait_for` является корутиной, поэтому ее нужно ожидать.
- `try...except asyncio.TimeoutError`: `wait_for` сигнализирует о провале с помощью исключения `asyncio.TimeoutError`, поэтому вызов почти всегда оборачивают в `try...except`.

Два возможных сценария

1. Сценарий 1: Успех (операция завершилась вовремя).

- `wait_for` запускает `some_coroutine()`.
- `some_coroutine()` успешно завершается до истечения 5 секунд и возвращает результат.
- `wait_for` перехватывает этот результат и возвращает его дальше.
- Ваша программа получает результат в переменную `result` и продолжает работу, не заходя в блок `except`.

2. Сценарий 2: Провал (тайм-аут).

- `wait_for` запускает `some_coroutine()`.
- Проходит 5 секунд, а `some_coroutine()` все еще "висит" на каком-то внутреннем `await`.
- `wait_for` видит, что время вышло. Он больше не ждет.
- Он немедленно вызывает исключение `asyncio.TimeoutError`.
- Выполнение переходит в ваш блок `except asyncio.TimeoutError`, где вы можете обработать эту ситуацию.

Что происходит с "зависшей" задачей?

Это очень важный момент. Когда `wait_for` вызывает `TimeoutError`, он не просто перестает ждать. Он также **запрашивает отмену** (вызывает `.cancel()`) для той задачи, которую он ждал.

Это значит, что "зависшая" корутина получит внутри себя исключение `CancelledError` (как мы обсуждали в прошлом уроке). Это позволяет ей корректно завершиться и освободить ресурсы, которые она могла захватить.

Ключевой вывод: `asyncio.wait_for` — это ваш "предохранитель". Он превращает неопределенное ожидание в понятную и обрабатываемую ошибку `asyncio.TimeoutError`, автоматически запрашивая отмену "зависшей" задачи.

Давайте напишем программу, которая продемонстрирует оба сценария работы `asyncio.wait_for()`: когда операция укладывается в отведенное время и когда нет.

Мы создадим одну корутину `slow_operation(delay)`, которая просто "спит" заданное количество секунд. Затем мы вызовем ее дважды с разными тайм-аутами.

Пример кода

Скопируйте и запустите этот код. Внимательно следите за выводимыми сообщениями и паузами.

Разбор полетов

1. **В первом сценарии** все просто: `slow_operation(1)` завершается за 1 секунду, что меньше тайм-аута в 2 секунды. `wait_for` успешно возвращает ее результат.
2. **Во втором сценарии** происходит самое интересное:
 - Запускается `slow_operation(3)`. Она начинает свое "ожидание" длиной в 3 секунды.
 - Но `asyncio.wait_for` параллельно запускает свой собственный "таймер" на 2 секунды.
 - Ровно через **2 секунды** таймер `wait_for` срабатывает. Он не ждет оставшуюся секунду.
 - Он немедленно вызывает `asyncio.TimeoutError` и **отменяет** задачу `slow_operation(3)`.
 - Именно поэтому вы **не увидите** сообщение " >> Медленная операция (3 сек) завершена." во втором сценарии. Задача была прервана до того, как успела дойти до этого `print`.

Этот пример наглядно показывает, как `asyncio.wait_for` дает вам полный контроль над максимальным временем ожидания любой асинхронной операции.

